

Site Exhibit Tasks

1. What you are building

Build a Python tool that turns site information into print-ready maps. The tool must use public data and open-source libraries.

There are only two kinds of map:

- Site Location: site outline on a topographic map.
- Site Plan: site outline on an aerial photo. Make one for the most recent year, and one for every older aerial-photo year you can find.

Alongside the maps, write a manifest: a JSON file that records every source and output used.

For this assignment, do not build the search-radius map or the adjoining-parcels map.

The basic flow is:

- Input: latitude/longitude, parcel numbers, or a boundary file.
- Find the parcel shapes and combine them into one site outline.
- Fetch the topo map and the aerial imagery for each year available.
- Draw the site outline, label, north arrow, scale bar, year, and title block.
- Save the pages as PDFs and check that the maps are correct.
- In Part B, let an AI agent choose the steps and react to failures. It produces the same maps.

Note: The AI does not perform geometry or draw maps. Your Python code does that. The AI only chooses tools, reads the results, and asks for help when needed.

2. What you are given

You will receive a test pack. It contains:

- Twenty site request files, in the JSON shape shown below.
- Parcel data for all twenty sites: parcel numbers, parcel shapes, and the combined site outline. The numbers and shapes are made up, so you can build and test without a network connection.
- Sample output maps, showing the layout your maps must match.
- A checklist of every element a map must contain, and a JSON schema for the manifest.
- A script that fetches real parcel numbers from county services when you are ready to use them.

Use the sample maps only as a visual example. Do not reuse their images.

A request follows this JSON shape:

```
{  
  "project_number": "PA-2026-0142",  
  "site_name": "Former Ridgeway Auto Service",  
}
```

```

"county": "Douglas County, NE",
"parcel_ids": ["0123456789", "0123456790"],
"latitude": null,
"longitude": null,
"boundary_geojson": null,
"figures": ["site_location", "site_plan", "historical"],
"output_dir": "./out/PA-2026-0142"
}

```

At least one of these must be provided: parcel_ids, latitude/longitude, or boundary_geojson.

What to install

```

pip install geopandas shapely pyproj contextily matplotlib
          matplotlib-scalebar rasterio requests
          pystac-client planetary-computer

```

Part A should use free public data and open-source libraries. Part B needs a model that supports function calling. You may run any open-source model.

Part A - Build the mapping pipeline

Task 1. Turn parcel numbers into one site outline

A site can contain several neighbouring parcels. Your job is to fetch each parcel shape and combine them into one site boundary.

1. Start with Douglas County, Nebraska. Find the Parcels dataset and its API.
 - Douglas County GIS data: <https://data.dogis.org/>
2. Write resolve_site(county, parcel_ids, coords). It should request the parcel geometry for every parcel ID.
3. Clean parcel IDs before searching. Handle spaces, dashes, and extra zeros. If one format fails, try reasonable alternatives.
4. Combine the parcel shapes with GeoPandas. Convert the geometry to UTM before measuring distances or areas.
 - GeoPandas: <https://geopandas.org/>
5. Two sites in the test pack are built to catch mistakes here. One is an L-shaped group of three parcels, so a bounding box is not the same as the outline. One has a 20 mm gap between two parcels, so the combined shape comes back in two pieces unless you choose a snapping tolerance. Record the tolerance you chose.
6. If latitude/longitude is also provided, confirm the point is inside the site outline. If only latitude/longitude is provided, find the parcel containing that point.
7. Return the final site outline as GeoJSON and record where it came from: parcel IDs, coordinates, or a boundary file.

Done when: This works for at least three counties. Multiple parcel IDs become one outline. A bad parcel ID returns a clear error that includes the ID instead of crashing.

Task 2. Fetch the topo map and aerial photos

Each map needs a background image: a topo map, a recent aerial image, or an older aerial image.

8. Write `get_layer(kind, bbox, year)`. Supported kinds are topo and aerial.
9. Use the USGS Topo service for topo tiles. contextily can fetch tiles from a URL.
 - USGS Topo service: <https://basemap.nationalmap.gov/arcgis/rest/services/USGSTopo/MapServer>
 - contextily documentation: <https://contextily.readthedocs.io/>
10. For aerial photos from 2010 onward, use NAIP through Planetary Computer and search it with the STAC API. For a quick recent image, the USGS imagery service is also acceptable.
 - NAIP dataset: <https://planetarycomputer.microsoft.com/dataset/naip>
 - Planetary Computer STAC API: <https://planetarycomputer.microsoft.com/api/stac/v1>
 - USGS Imagery Only service: <https://basemap.nationalmap.gov/arcgis/rest/services/USGSImageryOnly/MapServer>
11. For NAIP 2003-2009 and DOQ imagery from the 1990s, use EarthExplorer. A free EarthExplorer account is required for downloads; manual downloads are acceptable for test sites.
 - USGS EarthExplorer: <https://earthexplorer.usgs.gov/>
 - USGS guide to aerial imagery datasets: <https://www.usgs.gov/faqs/what-aerial-imagery-data-set-should-i-use-earthexplorer>
12. Write `list_years(bbox)`. It should return the aerial years available for that location. Do not assume every place has imagery every year. One test site is in Alaska, where NAIP has no coverage at all.
13. If an aerial photo has no map coordinates, skip that year and record it as missing. You are not asked to georeference scans.
14. Cache every downloaded file. When you run the same site again, the program should use the cache and make no network requests.

Done when: You can get a georeferenced image for an available year. A second run uses the cache. Deleting the cache causes it to download again. A location with no imagery returns an empty list instead of an error.

Task 3. Draw one reusable map frame

Every page should use the same layout. Build that layout once and reuse it for every map.

15. Review the sample maps and list every required visual element in `checklist.md`: image, red site outline, SITE LOCATION callout, north arrow, scale bar, year, and title block.
16. Write `render(layer, boundary, title, year)`. It should draw the background, red outline, callout, north arrow, scale bar, year, and title block.
 - matplotlib-scalebar: <https://github.com/ppinard/matplotlib-scalebar>
17. The scale bar must be based on real-world map width in metres, not pixel count.
18. Choose the map extent so the site fills roughly half the page, then round to a standard map scale. Choose the scale and the scale bar together. At 1:2,400 a 7.5 inch map frame covers only 1,500 feet, so a 0 to 2,000 foot bar will not fit. Use a shorter bar, or a smaller scale such as 1:6,000. Record the bar length you drew.

19. If imagery does not cover the complete page, fill the uncovered area with light grey and write UNMAPPED.
20. Save each page as a 300 DPI PNG and a US Letter PDF.

Done when: Every map has the same layout, every checklist item is present, and the printed scale bar is accurate to within 1%.

Task 4. Make the two current maps

21. Create Site Location: site outline on the topo map.
22. Create Site Plan: site outline on the most recent aerial image.
23. Create manifest.json. For each map, record the output files, source, photo date, projection, map scale, scale bar length, and source of the site outline.

Done when: One command creates both maps and the manifest for any test site.

Task 5. Make the aerial series for older years

Create the same site view for every older aerial year you can obtain.

24. Use list_years to find the available years. Render one page for each year using the Task 3 frame. Put the newest year first.
25. Combine all yearly pages into one PDF.
26. Record both available and missing years in the manifest.

Done when: The aerial PDF matches the sample layout for the years you found. Missing years are listed clearly.

Task 6. Validate the output

A map can look correct and still be wrong. Add automatic checks.

27. Write validate(output_dir). It should read the manifest and the generated maps.
28. Check that the site outline contains the input coordinates when coordinates were provided.
29. Check that the scale bar matches the real map width.
30. Check that every checklist.md element was drawn. Use the values recorded by render; do not inspect pixels for this check.
31. Fail the run if any parcel shape is marked synthetic, so the sample parcel data cannot end up in a finished map.
32. Create a JSON schema for manifest.json and validate the manifest against it.
33. Print pass/fail for every check and save a JSON validation report.

Done when: validate works on any output folder. If you deliberately break something, such as the projection or scale bar, validation fails clearly.

Task 7. Package the pipeline

```
exhibits generate site.json
exhibits batch sites/
exhibits validate out/PA-2026-0142/
```

34. Build the three commands shown above.
35. Add all 20 test sites as input files.
36. Write a README so a new developer can install and run the project from a fresh clone.
37. Write tests for `resolve_site`, for parcel ID cleaning, and for the scale bar.

Done when: All 20 sites run in one batch without manual steps, and a new machine can be set up and run the project in under 15 minutes.

Part B - Do the same job with an agent

Part A always follows a fixed pipeline. In Part B, an AI model decides which tool to use next, handles normal failures, and asks a human when it cannot safely continue.

Part B makes the same maps from the same data. It adds no new figures and no new sources. If Part B produces a different package from Part A for the same request, one of them is wrong.

Task 8. Turn pipeline functions into agent tools

38. Create tool definitions for `resolve_site`, `list_years`, `get_layer`, `render`, and `validate`. Each tool needs a name, a short description, and a JSON schema for its inputs.
39. Every tool must return either a valid JSON result or a valid JSON error. Do not expose exceptions or stack traces to the model.
40. Give errors a short code as well as a sentence, such as `not_found`, `no_coverage`, `unsupported_county`, or `invalid_request`.
41. Write a dispatcher that receives the requested tool name and inputs, then calls the correct Python function.
42. Keep calculations inside Python. The model chooses inputs and tools; your code performs geometry and measurements.

Done when: Any function-calling model can use the tool definitions, and you can test the dispatcher without a model.

Task 9. Build the agent loop

The loop should work like this: request -> model chooses a tool -> Python runs it -> result goes back to the model -> model chooses the next step -> repeat until complete.

Example:

```
request: "maps for plots 0123456789 and 0123456790 in Douglas County NE,
         older photos back to 1990"
```

```
-> resolve_site(...)           ok, 2.77 acres
```

```

-> list_years(...)          9 years found, 1994 missing
-> render("site_location")  ok
-> render("site_plan")      ok
-> render each older year   ok
-> validate(...)            pass
-> done: files + manifest

```

43. Choose a model that supports function calling. An open model through Ollama is acceptable.

- Ollama: <https://ollama.com/>

44. Implement the loop: send the request and tool definitions, execute the model tool call, send the result back, and continue until the model says the job is complete.

45. Log every step: requested tool, returned result, and run time.

46. Set a maximum number of steps so the loop cannot run forever, and a maximum number of retries per tool so it cannot repeat one failure.

47. Test three different request styles: parcel IDs; latitude/longitude only; and a boundary file requesting only Site Plan.

Done when: All three test requests produce the correct outputs through different tool sequences, and the log records each step.

Task 10. Ask a human instead of guessing

The agent must not confidently produce a map for the wrong site.

48. Stop and ask for help when no parcel matches, coordinates are outside every parcel, a county name is ambiguous, or no imagery exists for the requested years.
49. Return the question as JSON. Include what the agent tried, what it found, and what information it needs from a person.
50. Never silently substitute a nearby parcel or a different year.
51. Some failures are normal and the agent should carry on rather than ask. The Alaska site has no aerial coverage: the agent should still make the topo map, skip the aerial years, record the gap, and finish.

Done when: The project has one clear question/error format and no silent substitutions across the 20-site test batch.

Task 11. Show that the agent matches the pipeline

The agent is only correct if it produces what the fixed pipeline produces.

```

exhibits batch sites/ --mode pipeline --out out/a/
exhibits batch sites/ --mode agent    --out out/b/
exhibits compare out/a/ out/b/

```

52. Run all 20 sites through Part A, then again through Part B.
53. Compare the two manifests for each site: number of maps, years rendered, years missing, site outline area, projection, map scale, and scale bar length.
54. A different tool order is fine. A different set of maps is not. Explain any difference you find.

55. Record what the agent cost: steps taken, tool calls, retries, and run time compared with the pipeline.

Done when: All 20 sites produce matching manifests in both modes, and any difference is explained.

What to hand in

- A GitHub repository with the code, tests, and README.
- The output folders for all 20 sites in both modes, with manifests and validation reports.
- The comparison report from Task 11, and the agent log for at least three sites.
- A short note saying which sites produced complete output, which produced partial output, and why.

Reading

This section is reference material.

Site Location: Shows where the site is on a general/topographic map.

Site Plan: Shows the site on an aerial image.

parcel / plot: A piece of land recorded by local government. A site may contain several parcels.

county: A local government area within a US state. Parcel records are usually managed county by county.

site outline: The combined boundary of the parcel(s) making up the site. Draw it in red on each map.

USGS: The US government mapping agency.

topo map: A map showing roads, buildings, rivers, terrain, and other features.

NAIP: A US aerial-imagery program that usually photographs areas every two or three years. It covers the lower 48 states.

DOQ: Older USGS digital aerial imagery, mainly from 1987-2006.

tile service: A service that provides maps as many small image tiles.

ArcGIS REST: A web API often used by counties to publish parcel geometry and attributes.

STAC: A standard API for searching satellite and aerial imagery by location and date.

GeoJSON: A text/JSON format for geographic points, lines, and polygons.

coordinates: Latitude and longitude describing a location on Earth.

projection: A method for displaying the round Earth on a flat map.

UTM: A projection useful for accurate local distance and area measurements, in metres.

Web Mercator: The common map-tile projection. Do not use it for accurate measurement; convert to UTM.

bbox: Bounding box: the rectangle covered by a map or image request.

scale bar: The map ruler showing real ground distance.

callout: A label with an arrow pointing to a feature, such as SITE LOCATION.

manifest: A file recording outputs, sources, dates, projections, and other metadata.

function calling: A model returns the name of a tool/function and the arguments that should be passed to it.

dispatcher: The code that takes a tool name from the model and calls the matching Python function.

US units: 1 mile is about 1.6 km; 1,000 feet is about 305 m; 1 acre is about 4,047 square metres.

US Letter: 8.5 x 11 inch paper size.

300 DPI: Print-quality image resolution: 300 dots per inch.

More useful links

- All USGS National Map services: <https://www.usgs.gov/faqs/where-can-i-find-a-list-urls-national-map-services>
- Which EarthExplorer dataset covers which years: <https://www.usgs.gov/faqs/what-aerial-imagery-data-set-should-i-use-earthexplorer>
- Historical USGS topo maps: <https://ngmdb.usgs.gov/topoview/>
- US Census geocoder: <https://geocoding.geo.census.gov/geocoder/>
- OSMnx documentation: <https://osmnx.readthedocs.io/>